PHY F376: A study of hybrid classical-quantum algorithms

Week 4 (11/9/2026) - Generalizing the SQD Workflow and Results on a Noiseless Simulator

Khushi Pandey (2023B5A70951G)

Overview of This Week's Work

Building on last week's Hamiltonian-parsing code, this week I built and ran a full SQD workflow on the H2O CAS(4e,4o) Hamiltonian, then generalized the code so that it is not tied to this one molecule. I also switched from a hand-written subspace-diagonalization step to `qiskit-addon-sqd`'s own configuration-recovery and diagonalization routines, which turned out to require resolving a qubit-ordering mismatch I had noted but not yet addressed. So far I have only run this on a noiseless simulator; I have not yet obtained the API token needed to submit jobs to real hardware through my professor's classroom account.

Building a Circuit Without CCSD Amplitudes

Since this Hamiltonian came only as a Pauli list, without the underlying one- and two-electron integrals, I could not seed a LUCJ ansatz with CCSD amplitudes the way the N2 tutorial does. Instead I built a particle-number-conserving circuit from orbital-rotation gates and a Jastrow correlation layer, and optimized its parameters directly against the Hamiltonian I already had, in place of the classical amplitude-seeding step. I also found that `qiskit-addon-sqd` has a qubit-Hamiltonian version of SQD (`configuration_recovery`, `subsampling`, and `qubit.solve_qubit`) that works directly on a Pauli operator rather than requiring the integral tensors, which meant I could use the library's real self-consistent configuration recovery instead of the simplified version I had written by hand previously.

Resolving the Qubit-Ordering Mismatch

Using this part of the library exposed the qubit-ordering issue directly, since its functions assume qubits are grouped by spin (all alpha orbitals first, then all beta), while the given Hamiltonian numbers qubits with alpha and beta interleaved. I fixed this properly this time, using `SparsePauliOp.apply_layout` to relabel the qubits into the expected ordering, and confirmed the ground-state energy was unchanged to numerical precision after the relabeling, without which the library's diagonalization routines would silently have produced incorrect results.

Making the Code General-Purpose

Since the project will eventually involve other Hamiltonians, I rewrote the parsing and workflow code so it does not hard-code anything specific to water: the qubit count is inferred directly from the file, and the electron count, spin, and reference energy are read from the header comments rather than typed in by hand. The qubit-ordering relabeling was also generalized into a formula for any number of orbitals, rather than the fixed permutation derived only for this 8-qubit case. In principle, running this on a different Hamiltonian should now only require changing the file path, as long as it follows the same header format.

Results on the Noiseless Simulator

Running the full workflow on the ideal simulator, the optimized ansatz alone sat about 18.8 mHa above the reference energy, which is expected given it is a fairly simple circuit not seeded by any correlated classical calculation. After self-consistent configuration recovery and subspace diagonalization, the energy converged to within about 3.3-3.9 mHa of the exact reference across iterations, a clear improvement over the ansatz alone and closer to, though not quite at, chemical accuracy. This is broadly consistent with the pattern seen in the N2 tutorial, where the noiseless run also converged quickly and stably.

Concluding Remarks

This week's main outcome is a working, general SQD pipeline that produces a sensible convergence curve on real chemistry data, which I take as reasonable evidence that the qubit-ordering fix and the switch to the library's own recovery routines were both done correctly. I have not yet run this on real hardware, since I am still setting up API access to my professor's classroom account; once that is in place, the next step is to run the same workflow on actual hardware and compare its convergence and final error against this noiseless result.